Data Driven Computer Animation

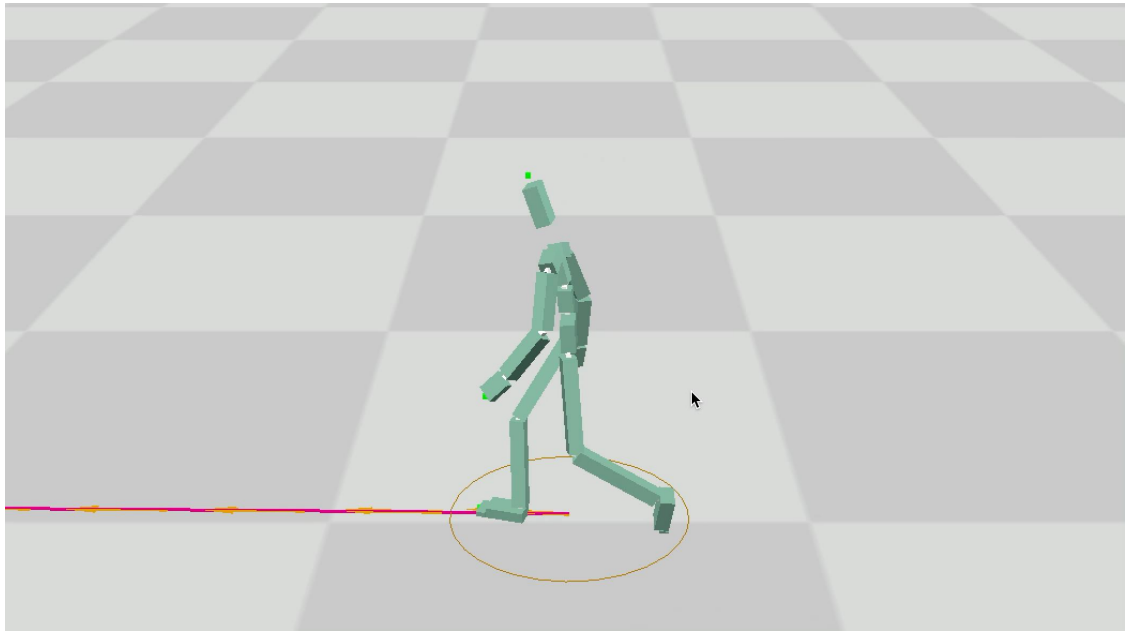
HKU COMP 7508

Tutorial 4: Real-time Character Control

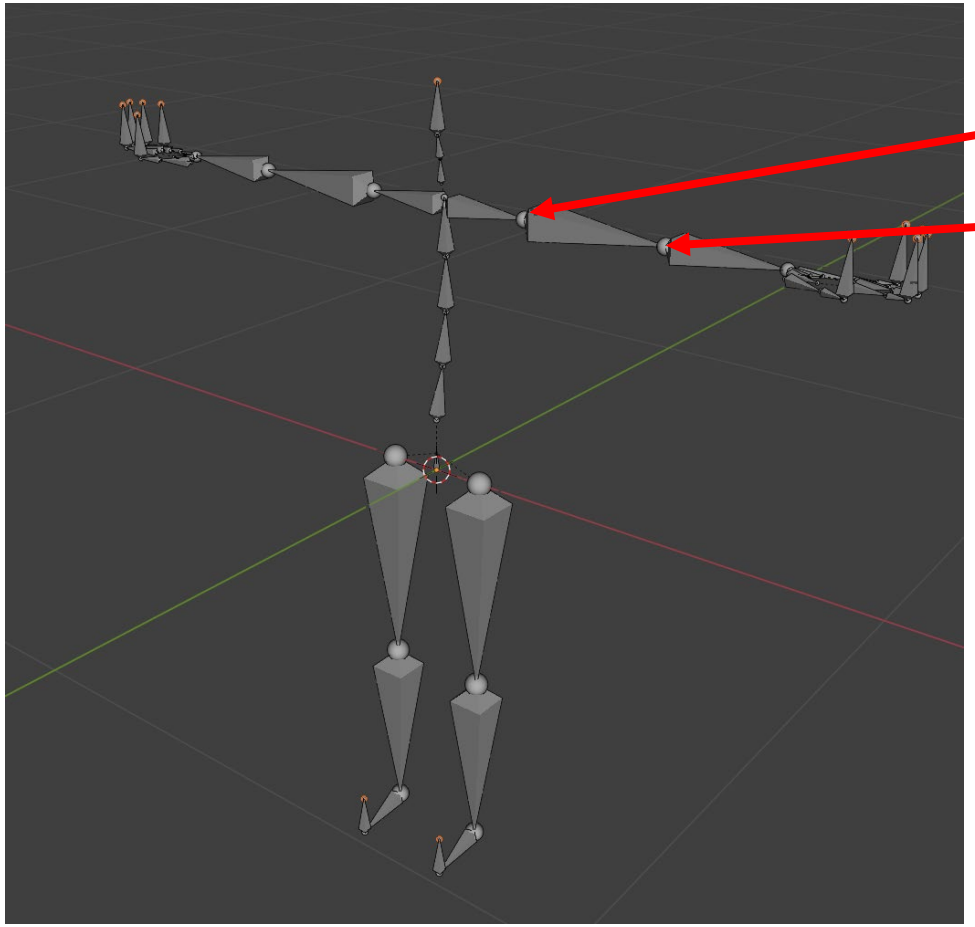


Agenda

1. Review on Assignment 1
2. The overview of real-time character control
3. State machine
4. Motion matching



Review on Assignment 1 (part 1)



$$p_i^{\text{world}} = p_j^{\text{world}} + \text{offset}_i$$

```
if parent_idx == -1:
    global_joint_position[joint_idx] = joint_offsets[joint_idx]
else:
    global_joint_position[joint_idx] = global_joint_position[parent_idx] + joint_offsets[joint_idx]
```

“Edit Mode” of an armature in blender

Review on Assignment 1 (part 2)

Review homogeneous transform matrix

Compose a homogeneous transform matrix by translation, rotation and scale

$$\mathbf{M} = \mathbf{T} \cdot \mathbf{R} \cdot \mathbf{S}$$

Through the order of scale, rotate and translate, we can write the transform matrix as follows:

$$\mathbf{M} = \begin{bmatrix} \mathbf{s}_x \cdot \mathbf{R}_{(:,1)} & \mathbf{s}_y \cdot \mathbf{R}_{(:,2)} & \mathbf{s}_z \cdot \mathbf{R}_{(:,3)} & \mathbf{p} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Review on Assignment 1 (part 2)

Understand “local transformation”

Rotate your shoulder, your elbow and hand also rotates.

Define an “unchanged” **local transform** for the elbow and hand, compute the **global transform** through the hierarchy

$$\mathrm{M}_{elbow}^{\text{world}} = \mathrm{M}_{shoulder}^{\text{world}} \cdot \mathrm{M}_{elbow}^{\text{local}}$$
$$\mathrm{M}_{hand}^{\text{world}} = \mathrm{M}_{elbow}^{\text{world}} \cdot \mathrm{M}_{hand}^{\text{local}}$$

After obtaining the **global transform** homogeneous matrices, we can extract the global position member from the matrices:

$$\mathbf{M} = \begin{bmatrix} \mathbf{s}_x \cdot \mathbf{R}_{(:,1)} & \mathbf{s}_y \cdot \mathbf{R}_{(:,2)} & \mathbf{s}_z \cdot \mathbf{R}_{(:,3)} & \boxed{\mathbf{p}} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Global position

Review on Assignment 1 (part 2)

The computation

Compose the “local transform” matrix in TRS order (BVH format don’t have scale so we skip it)

$$\mathbf{M}_{elbow}^{\text{local}} = \begin{bmatrix} \mathbf{R}_{elbow}^{\text{local}} & \mathbf{p}_{elbow}^{\text{local}} \\ 0 & 1 \end{bmatrix}$$

Bring this “detailed” version of transform matrix into out formula:

$$\mathbf{M}_{elbow}^{\text{world}} = \mathbf{M}_{shoulder}^{\text{world}} \cdot \mathbf{M}_{elbow}^{\text{local}}$$

$$\begin{bmatrix} \mathbf{R}_{elbow}^{\text{world}} & \mathbf{p}_{elbow}^{\text{world}} \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} \mathbf{R}_{shoulder}^{\text{world}} & \mathbf{p}_{shoulder}^{\text{world}} \\ 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} \mathbf{R}_{elbow}^{\text{local}} & \mathbf{p}_{elbow}^{\text{local}} \\ 0 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} \mathbf{R}_{shoulder}^{\text{world}} \cdot \mathbf{R}_{elbow}^{\text{local}} & \mathbf{R}_{shoulder}^{\text{world}} \cdot \mathbf{p}_{elbow}^{\text{local}} + \mathbf{p}_{shoulder}^{\text{world}} \\ 0 & 1 \end{bmatrix}$$

```
for joint_idx, parent_idx in enumerate(joint_parents):
    parent_orientation = R.from_quat(global_joint_orientations[:, parent_idx, :])
    rotated_vector = parent_orientation.apply(joint_positions[:, joint_idx, :])
    global_joint_positions[:, joint_idx, :] = global_joint_positions[:, parent_idx, :] + rotated_vector
    global_joint_orientations[:, joint_idx, :] = (parent_orientation * R.from_quat(joint_rotations[:, joint_idx, :])).as_quat()
```

Review on Assignment 1 (part 3)

Cyclic Coordinate Descent - Inverse Kinematics

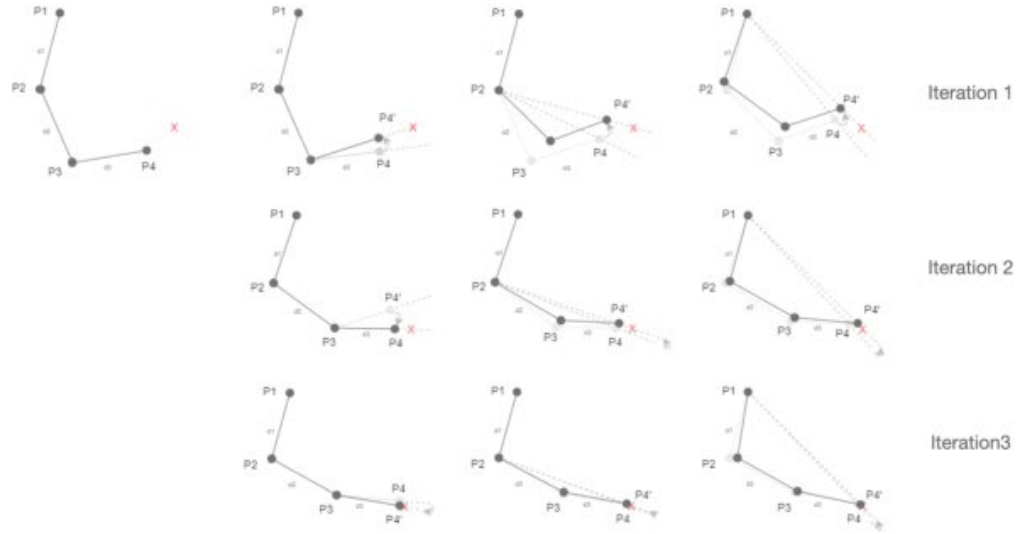
CCD-IK

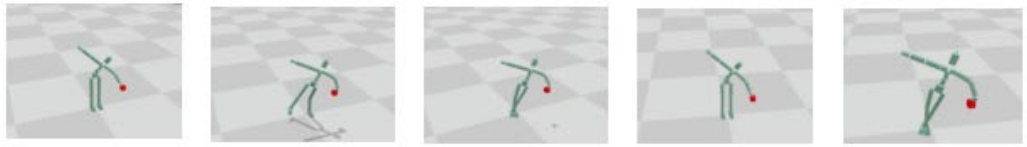
```
vec_cur2end = norm(chain_positions[end_idx] - chain_positions[current_idx])
vec_cur2tar = norm(target_pose - chain_positions[current_idx])

axis = norm(np.cross(vec_cur2end, vec_cur2tar))
rot = np.arccos(np.vdot(vec_cur2end, vec_cur2tar))

if np.isnan(rot):
    continue

rotate_vector = R.from_rotvec(rot * axis)
chain_orientations[current_idx] = rotate_vector * chain_orientations[current_idx]
```





Review on Assignment 1 (part 3)

https://drive.google.com/file/d/1NivTs93H0zGLK4KA2HKo6Q5HxgeVXqCU/view?usp=drive_link

Review on Assignment 1

We now have some basic knowledge about motion data

Let’s dive deeper to actual animation processing

Some materials are from:



Game Developers Conference (GDC)

1. https://www.gdcvault.com/play/1026968/-Genshin-Impact-Building-a
2. https://www.gdcvault.com/play/1025389/Character-Control-with-Neural-Networks

Animation

We captured motion clips, label them, and play them in different order



How to make the animation to be controllable?

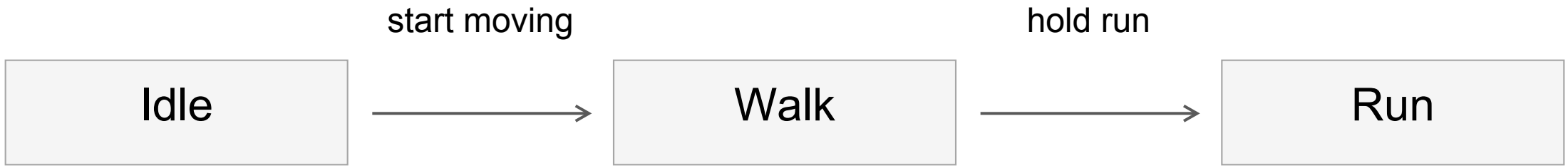
The character should reflect to different control signals

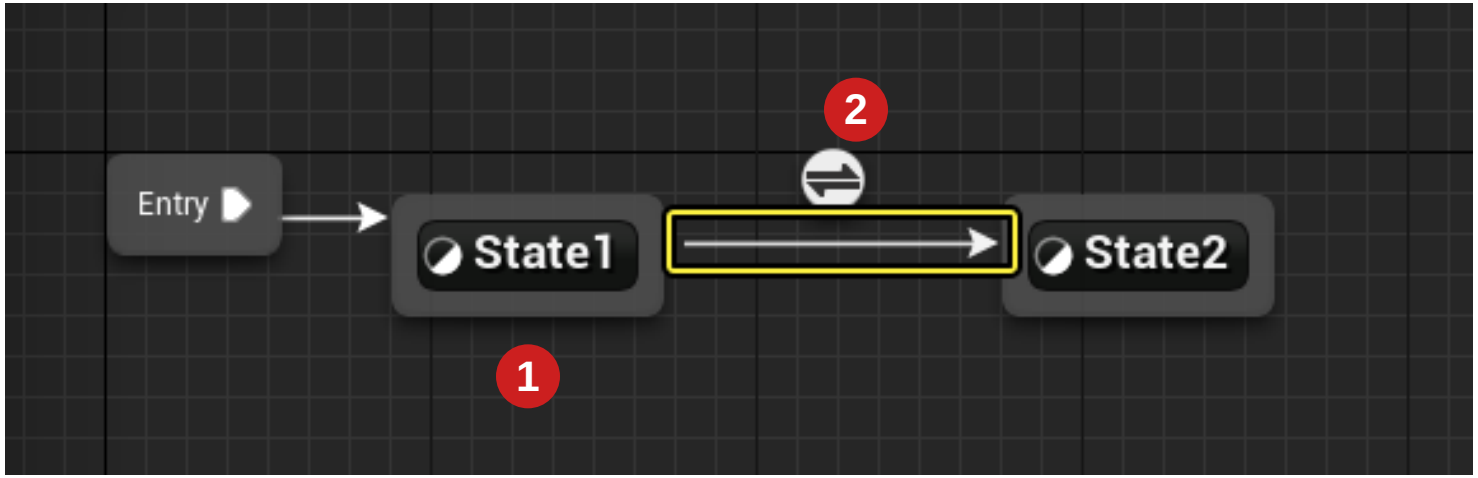
- from keyboard
- gamepad



State Machine: states and rules

Start with a simple example: the input decides which behavior is active.





1 State = behavior

Defines the animation to play, e.g. Idle or Run.

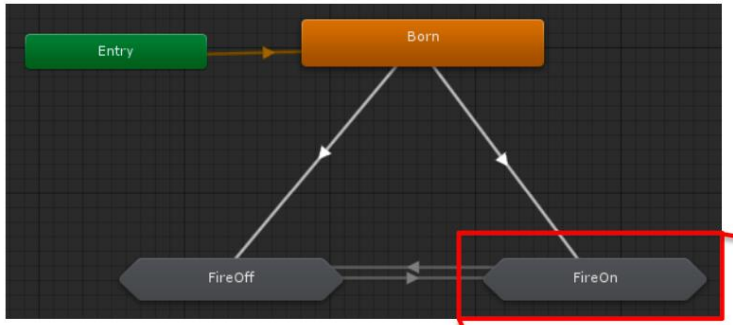
2 Transition = rule

Decides when to switch and blend between states.

Useful for explicit behavior control, such as deciding when a jump or attack is allowed.

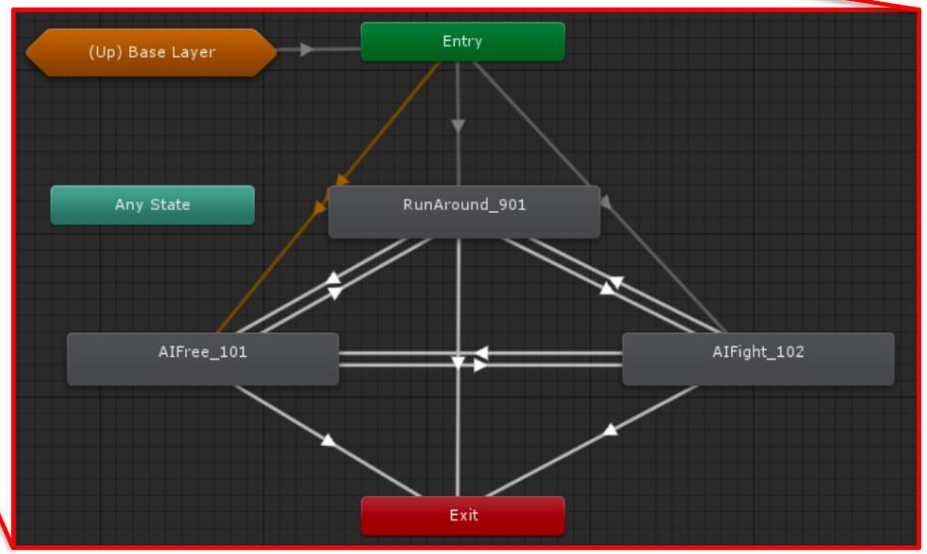
Image: Epic Games, State Machines (Unreal Engine documentation); numbered annotations added.

The states are managed by ‘node’



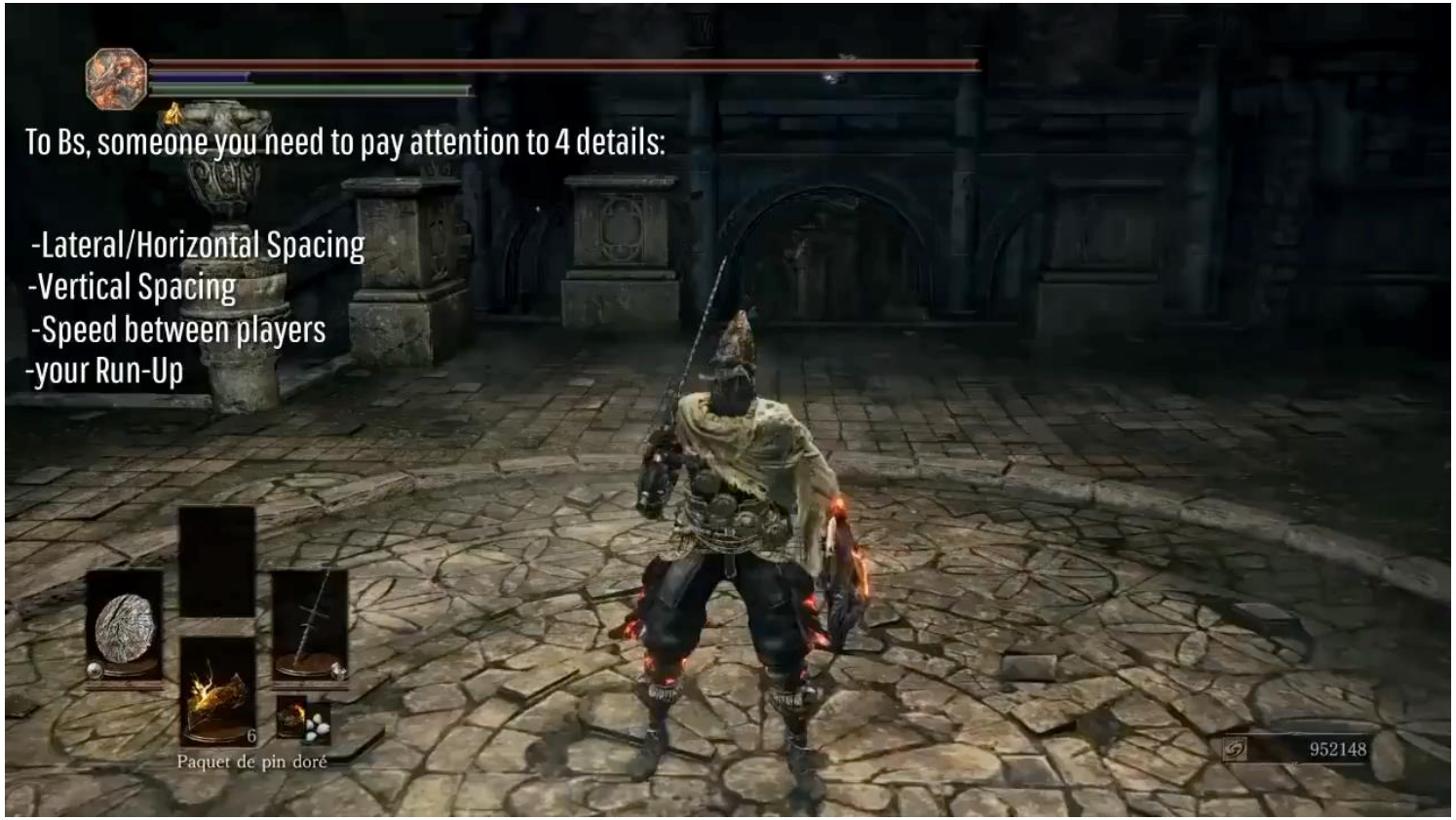
example of fire slime



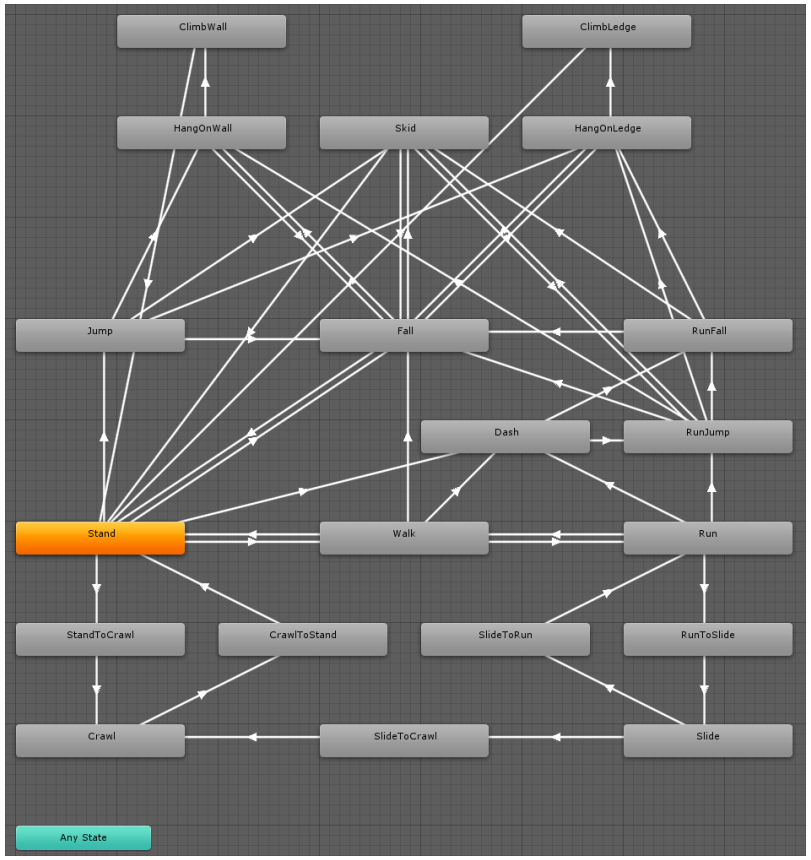


- A Layered System
- Node can be connected to any other node

Node system contains lots of transition



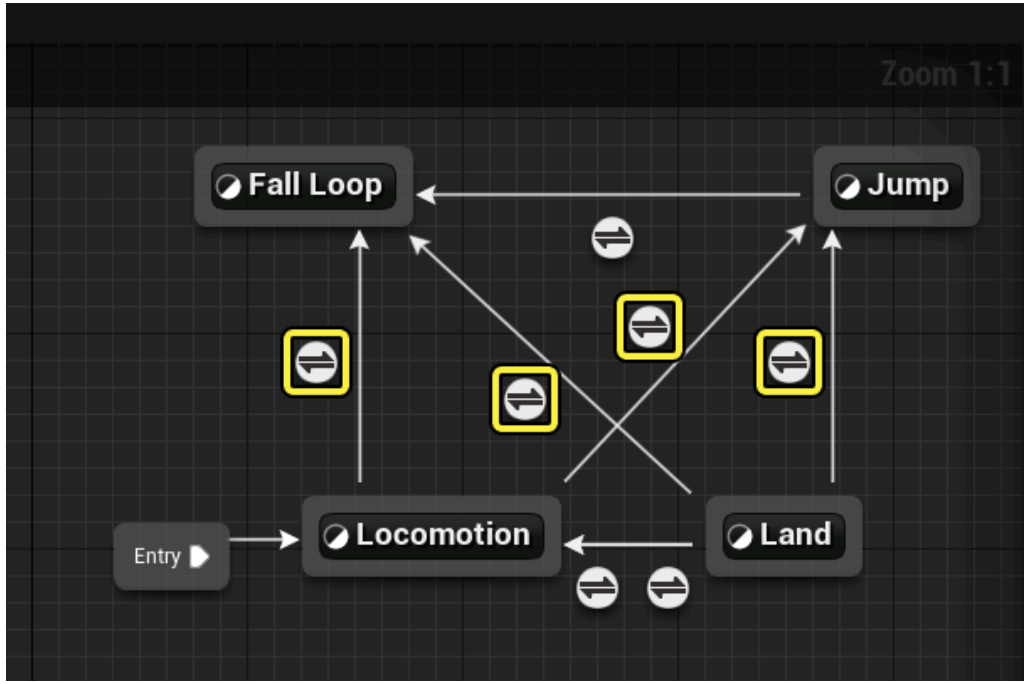
Controlling system is a combination for lots of motion clips



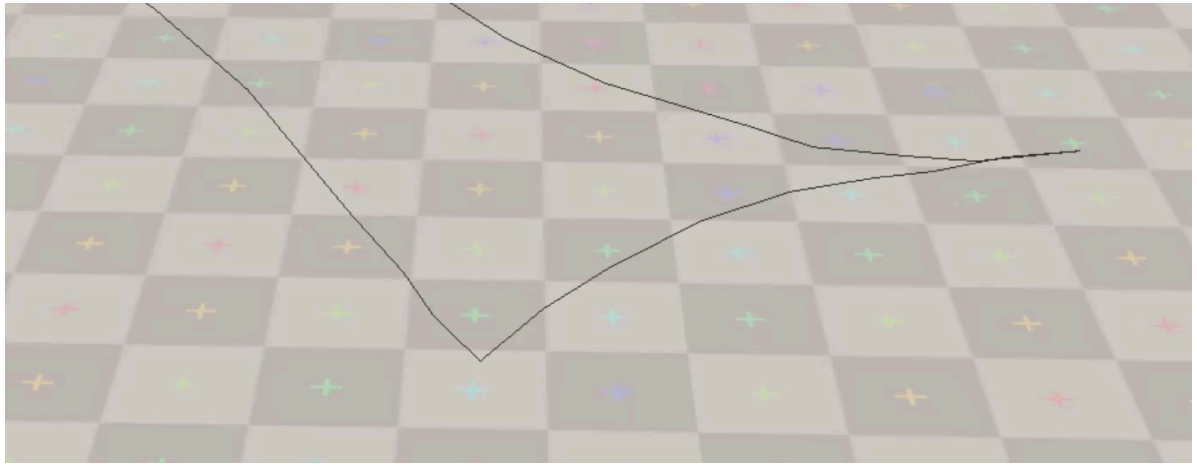
Character State Machine

More control requests, more decisions

A few states are easy to manage. More behaviors mean more choices and rules.



What if the player changes the path?



Gentle turn? Sharp turn? Slow down?

Can recorded motion examples help us choose what to play next?

We want smooth motion that follows the changing input, with less manual selection.

Left: Epic Games, State Machines. Right: Ubisoft La Forge, path-following example (video still, cropped).
Path example: Ubisoft La Forge, Introducing Learned Motion Matching (2020).

GDC talk by Daniel Holden

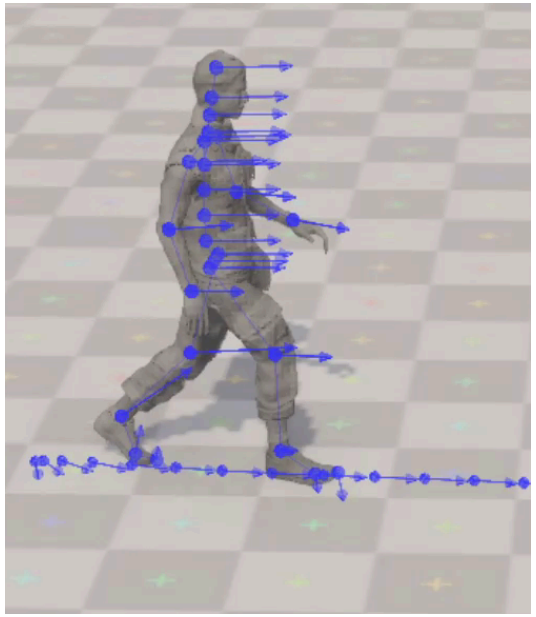
https://www.youtube.com/watch?v=o-QLSjSSyVk

PDF download: https://www.gdcvault.com/play/1025389/Character-Control-with-Neural-Networks

Motion Matching: choose from examples

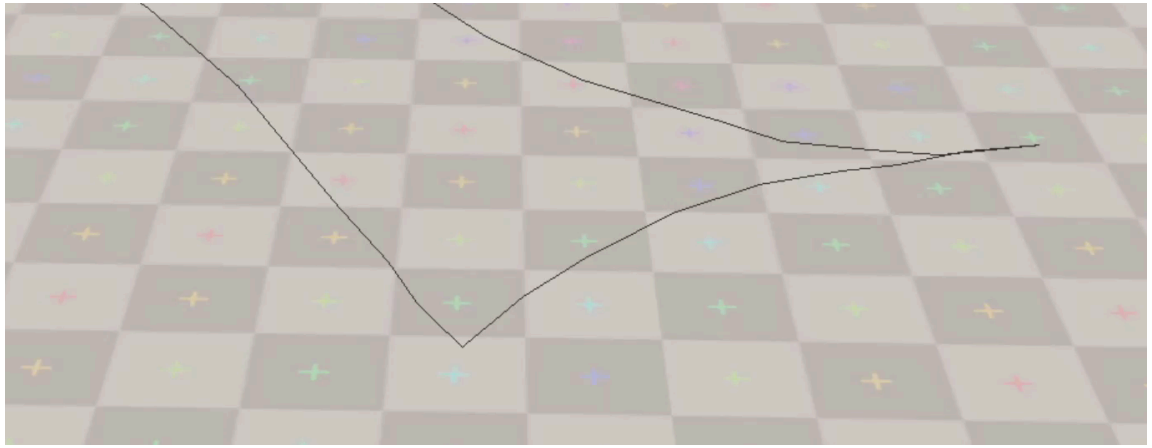
The character is moving; the player asks it to turn left. What should we play next?

1 Match the current motion



Similar pose and movement help make the switch smooth.

2 Match the desired future path



Choose motion that goes where the player wants.

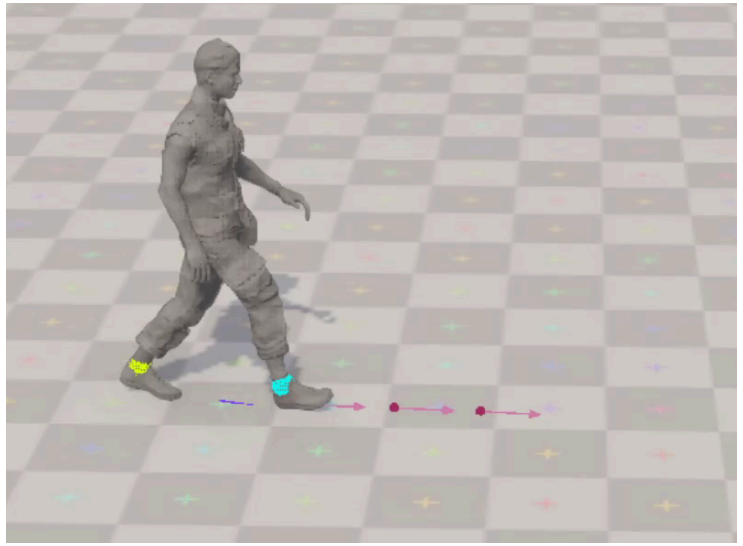
Search recorded frames for both; continue from a good match and blend when switching.

This is Motion Matching: search again as input changes, for responsive movement.

Images: Ubisoft La Forge, Introducing Learned Motion Matching (2020); cropped video stills.

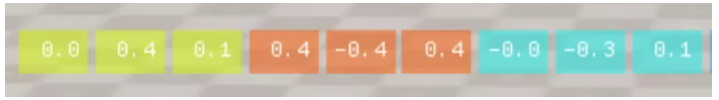
How can we search for matching motion?

Features are numbers describing motion and intent, so we can compare candidates.



From visible motion to numbers

Foot markers: positions and velocities.
Ground arrows: future path and facing.



Build the query from current motion and player intent; search for similar numbers.

These are the variables in Query(Dataset, variables), which we examine next.

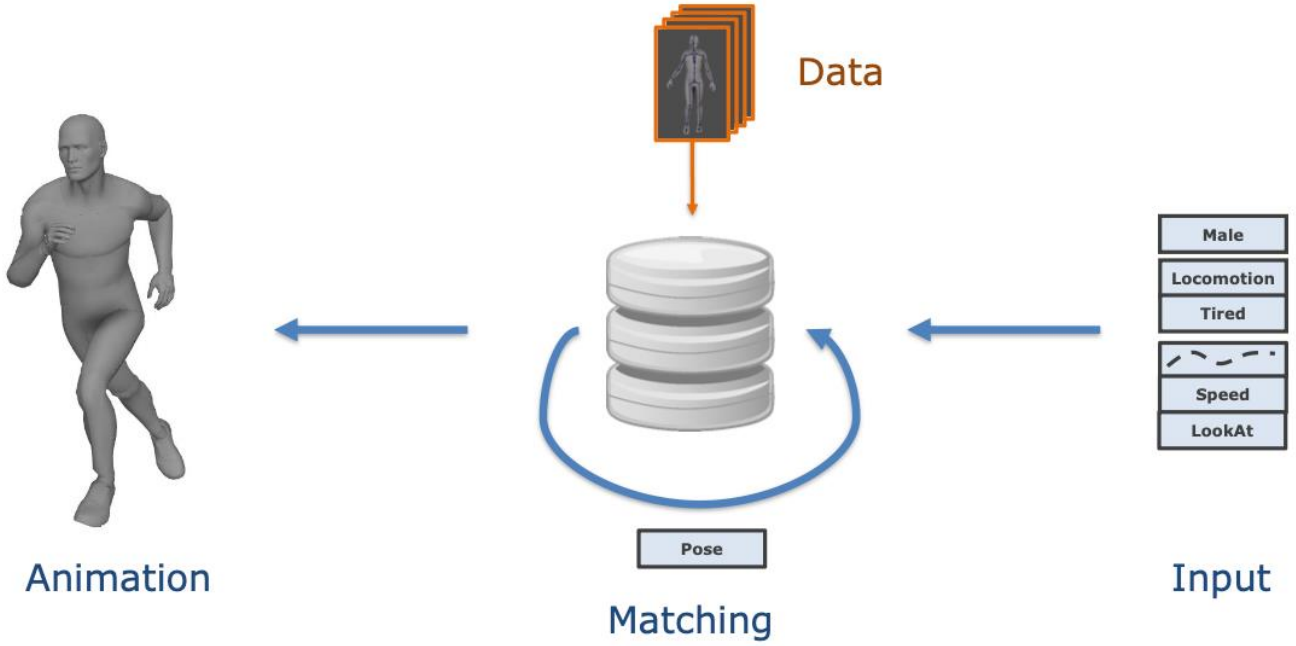
State Machine: rules select states. Motion Matching: features select recorded frames.

They can work together: a state machine can choose when to use motion matching.

Related idea: Motion Graphs pre-connect compatible recorded motion segments.

Images: Ubisoft La Forge (2020); crops from the same feature-visualization frame.
Motion Graphs: Kovar, Gleicher & Pighin (2002).

Programming Details



Animation = Query(Dataset, variables)

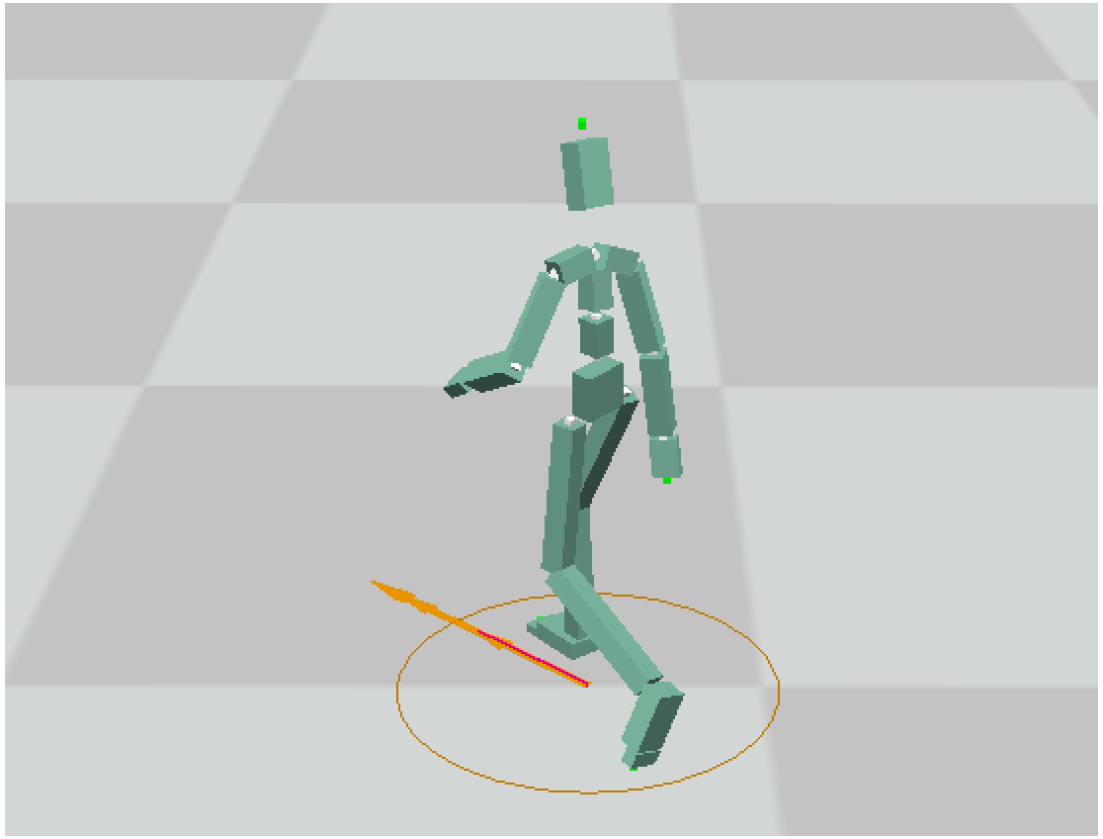
Select animation where variables=xxxxx from dataset

Feature Design

For a motion sequence with n frames

Each frame has its own features:

- Position
- Rotation
- Positional/Angular Velocity
- Future Position/Rotation after 20/40/60 frames



When we control the character

Each frame also has its own features:

- Position

- Rotation

- Positional/Angular Velocity

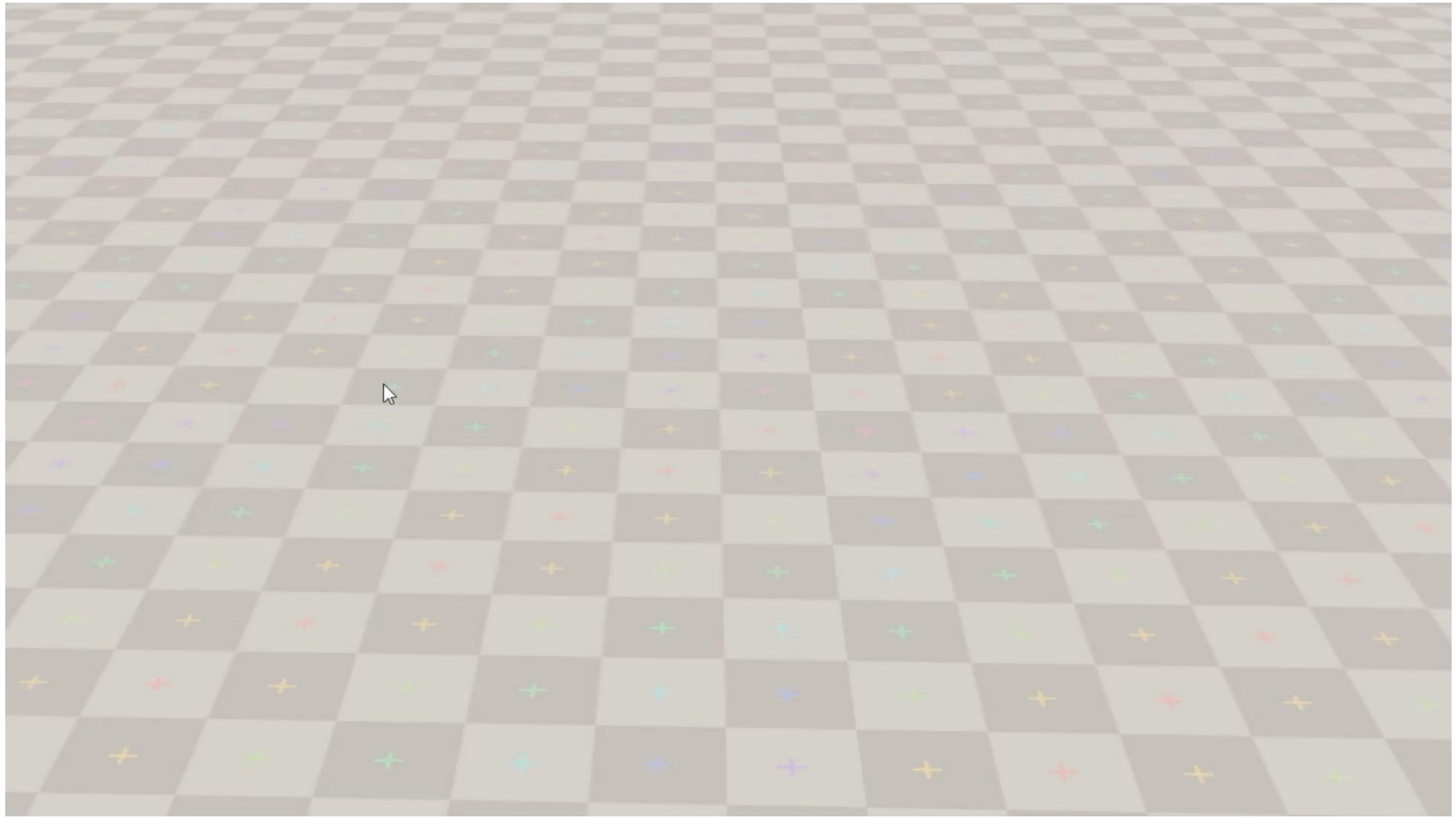
- Future Position/Rotation after 20/40/60 frames (decided by controller)

Step1: Give control signals

WALK

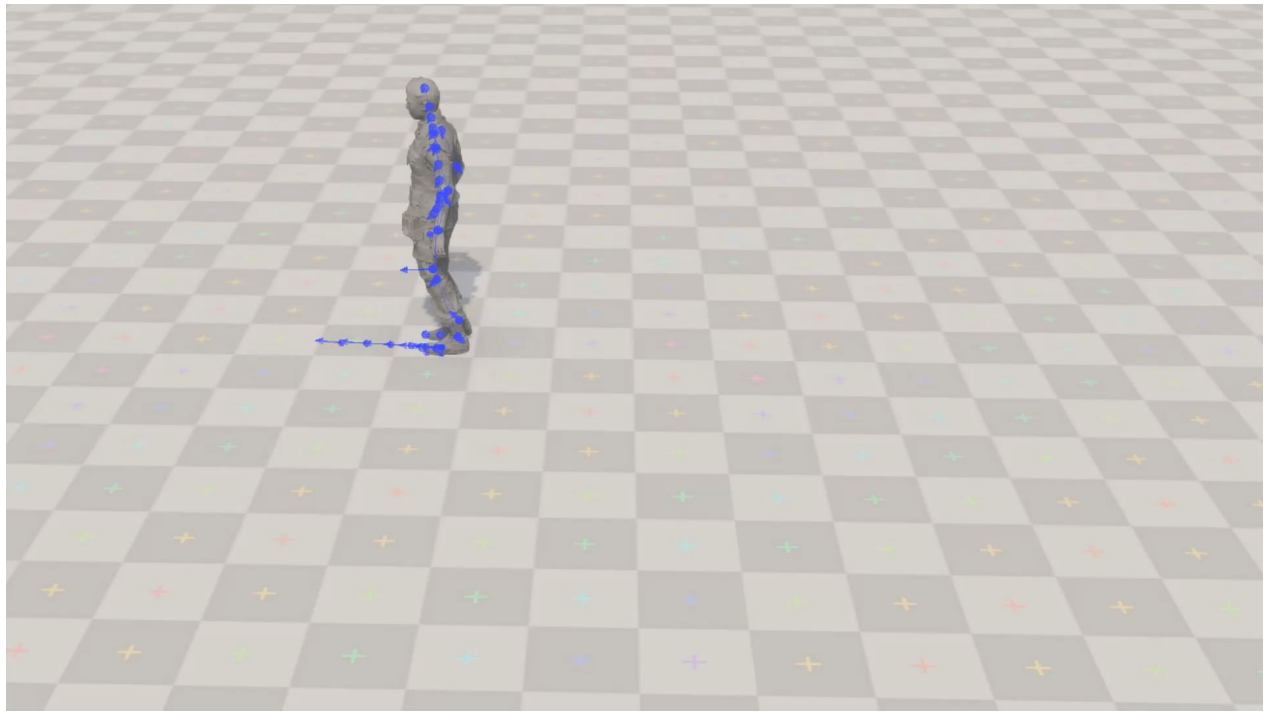
RUN

JUMP



FEATURES

Step2: Analyze the motion data



```
94          # Extract the data terms
95          pos, rot = forward_kinematics_with_channel(self.joint_parent, self.joint_channel, sel
96          rot = align_quat(rot, False)
97          vel = np.zeros_like(pos)
98          vel[1:] = (pos[1:] - pos[:-1])/self.dt
99          vel[0] = vel[-1]
100         avel = np.zeros_like(vel)
101         avel[1:] = quat_to_avel(rot, self.dt)
102         avel[0] = avel[-1]
```

We know:

1.Joint position

2.Joint velocity

3.Joint rotations

4.Joint angular velocity

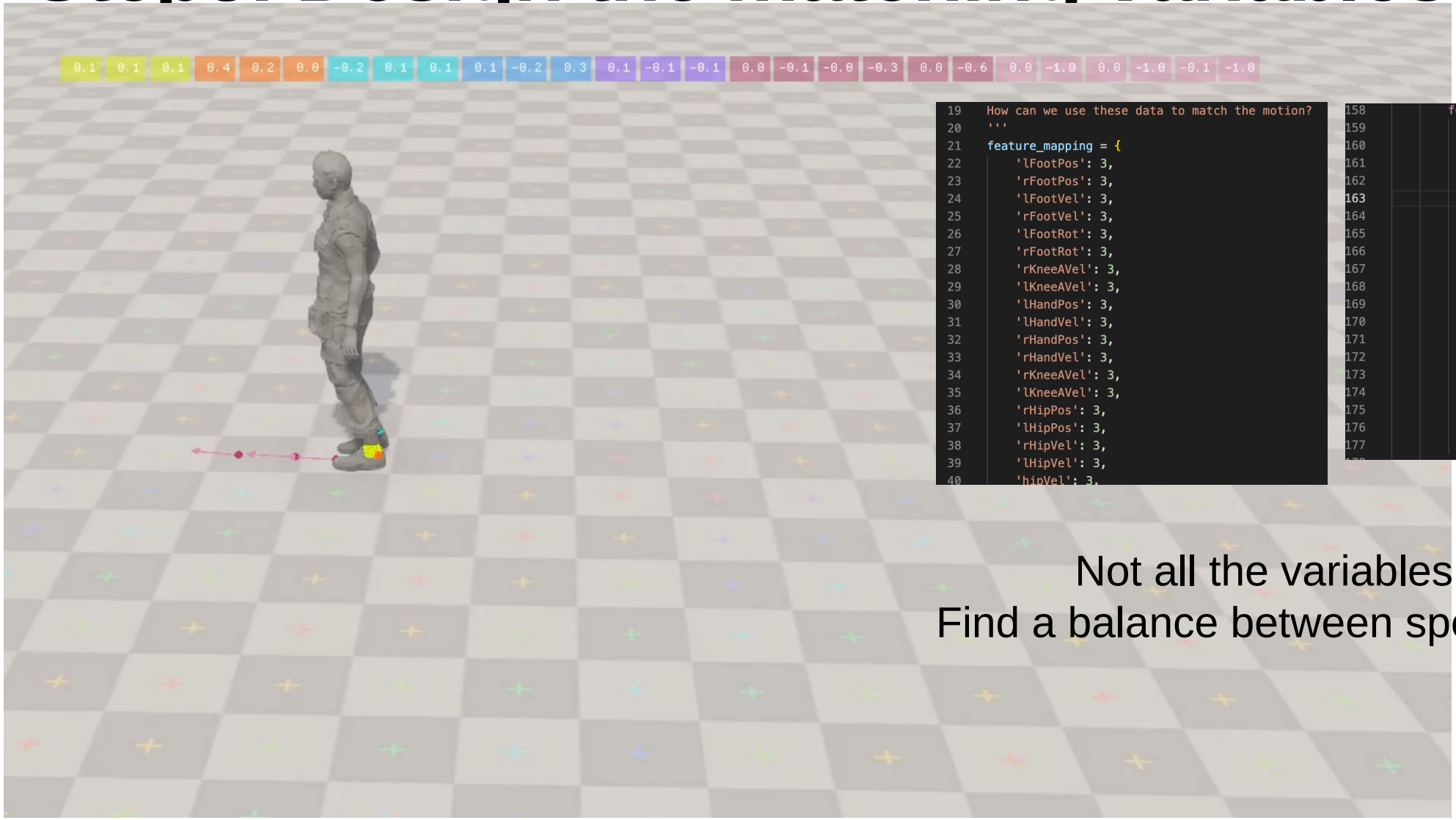
…

relative position

waving timing

…

Step3: Design the matching variables




```
How can we use these data to match the motion?
'''
feature_mapping = {
    'lFootPos': 3,
    'rFootPos': 3,
    'lFootVel': 3,
    'rFootVel': 3,
    'lFootRot': 3,
    'rFootRot': 3,
    'rKneeAVel': 3,
    'lKneeAVel': 3,
    'lHandPos': 3,
    'lHandVel': 3,
    'rHandPos': 3,
    'rHandVel': 3,
    'rKneeAVel': 3,
    'lKneeAVel': 3,
    'rHipPos': 3,
    'lHipPos': 3,
    'rHipVel': 3,
    'lHipVel': 3,
    'hipVel': 3,
```

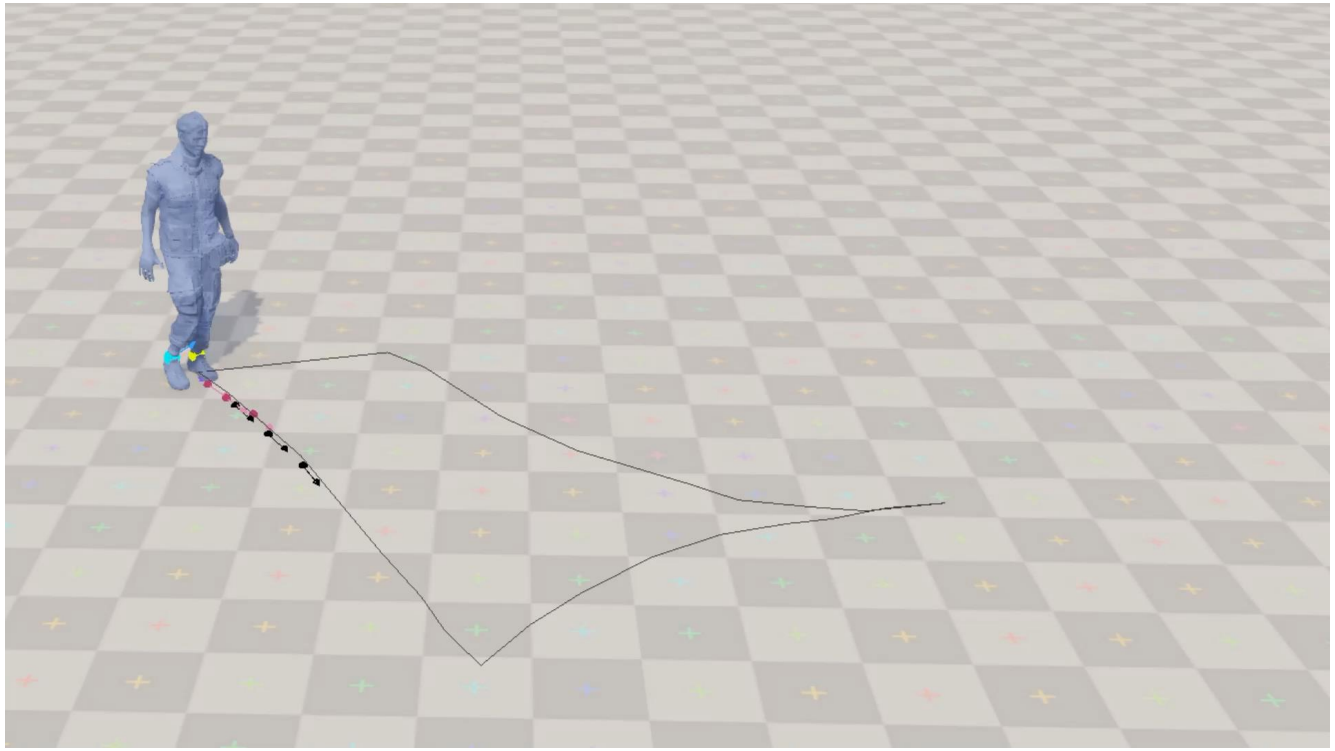
```
    for feature_name in self.feature_names:
        '''
        Extract the position:
            features.append(self.extract_offset(roo
        Extract the rotation:
            features.append(self.extract_rotation(r
        Extract the velocity:
            features.append(self.extract_vel(root_r
        Extract the future position:
        '''
        ##### Code Start #####

        if feature_name == 'lFootPos':
            features.append()
        elif feature_name == 'lFootVel':
            features.append()
        elif feature_name == 'trajectoryPos2D':
            features.append()

        ##### Code End #####
```

Not all the variables is needed
Find a balance between speed and accuracy

Step4: Matching



```
259             normalized_query = self.db.normalize_features(query_feature)
260
261             # Do the query
262             idx = self.db.query_tree.query(normalized_query.reshape(1,-1), k=1)[1][0]
```

Because we only use small dataset
We also store it with OC-tree
So the searching is fast

Programming Details

Prepare:

- A motion dataset
  - Oc-tree to accelerate the searching
- A set of feature variables
- A animation system to play the animation

The dataset and animation are prepared in this assignments

you only need to care the variables design

What you need to do:

Design the matching feature and its weights

```
def main():
    viewer = SimpleViewer()
    controller = Controller(viewer)

    selected_feature_names = ['trajectoryPos2D', 'trajectoryRot2D']
    selected_feature_weights = [1, 1]

    # selected_feature_names = ['lFootPos', 'rFootPos']
    # selected_feature_weights = [1, 1]

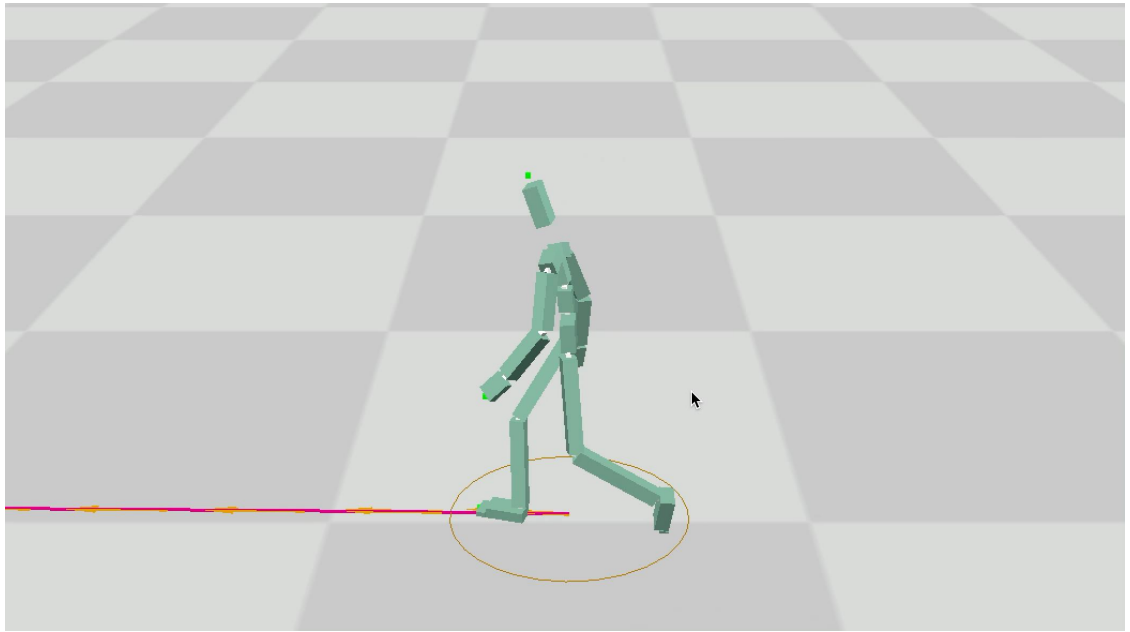
    assert len(selected_feature_names) == len(selected_feature_weights)

    character_controller = CharacterController(viewer, controller, selected_feature_names, selected_feature_weights)
    task = InteractiveUpdate(viewer, controller, character_controller)
    viewer.addTask(task.update)
    viewer.run()
    pass
```

- Some features are pre-defined. (L11~L35, L135~L197)
- If you want to add you new features, update the code around (L11~L35, L135~L197)

Desired Results

1. Less variables, and better performance (total 15%, 22% - your_variable_num)
2. System analyzation (10%) about variable selection, future frame range, etc.



Overview

- part1_key_framing (30%)
    - - Linear interpolation (10%); Slerp Interpolation (15%)
    - - Report the different performance by giving different numbers (5%)
- part2_concatenate (35%)
    - - Define the search window (10%); Calculate the sim_matrix (10%); Find the real_i and real_j (10%); The shifting on the root joint position (5)
- part3_motion_matching (25%)
    - variable terms (15% <- 22% - your_variable_num) + System analyzation (10%)
- Report (8%) + 2 videos (2%)
    - Including necessary experiment results by different parameters (4%) and your thinking(4%) for how to produce high quality motions

Assignment 2 deadline: 18 November 2026, 23:59

Please note: Late submissions will be subject to a 10% deduction per day and will only be accepted for up to 7 days following the deadline.